

FASE 5

Robótica móvil: diseño, construcción y autonomía

Esta fase reúne proyectos de integración en los que el estudiante diseña, construye, programa, prueba y documenta robots móviles. La práctica del robot esquiva obstáculos se incorpora como cierre integrador.

Actividad	Proyecto
Práctica 17	Robot Sumo 4WD de 1 kg con Arduino
Práctica 19	Conociendo y diseñando mi Robot Soccer
Práctica 20	Componentes del Robot Soccer - Parte 1
Práctica 20.1	Marcador inteligente para Robot Soccer
Práctica 21	Cómo está construido nuestro Robot Soccer
Práctica 22	Robot autónomo esquiva obstáculos
Ubicación curricular	Las prácticas 17, 19, 20, 20.1 y 21 dejan de presentarse en la fase anterior y forman parte de esta nueva Fase 5. La práctica 18 permanece fuera de esta agrupación.

Práctica 22

Robot autónomo esquiva obstáculos

Reto

Construir un robot que mida la distancia frontal, se detenga ante un obstáculo, explore ambos lados y elija una maniobra segura.

Aprendizajes esperados

- Relacionar entradas (sensor ultrasónico), procesamiento (Arduino) y salidas (servo y motores).
- Construir un chasis estable con materiales disponibles y cableado seguro.
- Programar la lógica con bloques en PictoBlox o trabajar con el código C++ generado.
- Calibrar distancia, velocidad y tiempos de giro mediante pruebas documentadas.

Producto esperado

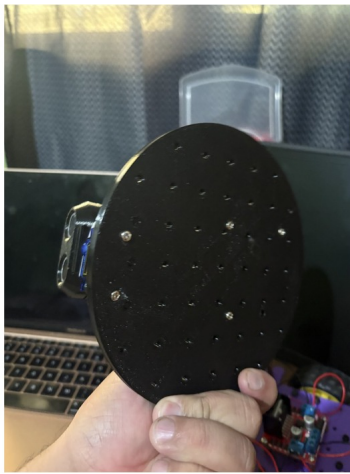
Robot autónomo funcional o prototipo verificable, programa copiable, tabla de pruebas, video breve y página individual de Google Sites con proceso, errores, correcciones y reflexión.

Duración sugerida: 3 a 4 sesiones

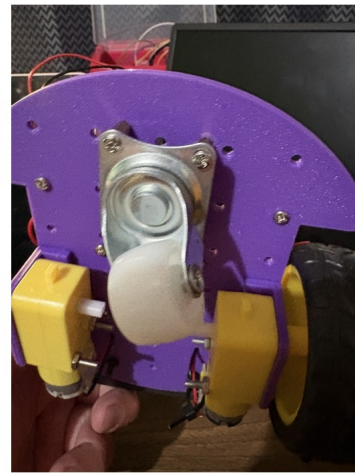
El chasis: dos caminos válidos

Importante

No es necesario imprimir el chasis en 3D. La impresión 3D es una opción, no un requisito. Puede utilizarse una tabla delgada, MDF, acrílico reciclado o cartón rígido, siempre que el montaje sea estable y seguro.



Base superior perforada: ejemplo de pieza impresa en 3D.

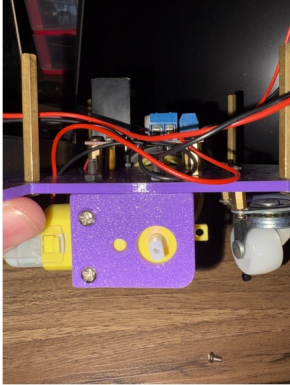
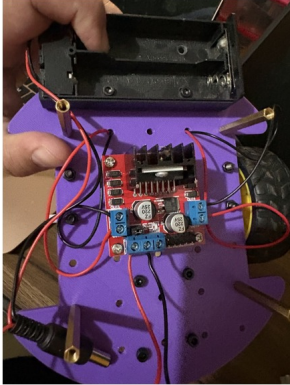
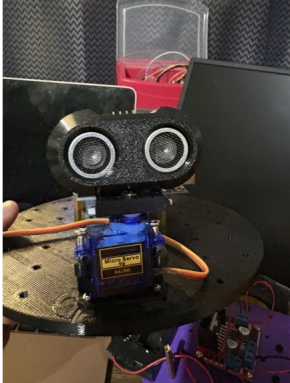
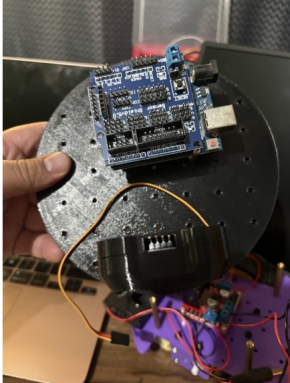


Vista inferior: motores y rueda loca fijados a la base.

Criterios mínimos para cualquier material

- La base no debe flexionarse hasta tocar las ruedas.
- Motores, batería y tarjeta deben quedar firmemente sujetos.
- No deben quedar puntas, tornillos o cables expuestos que causen cortos.
- El sensor debe mirar al frente y poder girar sin rozar.

Armado real: secuencia recomendada

	
1. Instala motores, ruedas y rueda loca; comprueba el giro libre.	2. Fija el puente H y ordena los cables de potencia.
	
3. Centra el servo a 90° y monta el sensor ultrasónico.	4. Coloca Arduino o el shield en la placa superior sin forzar conexiones.
Antes de energizar Revisa polaridad, continuidad de tierra común (GND), ausencia de cables sueltos y que ninguna rueda esté bloqueada.	

Conexiones de referencia

Adapta el diagrama al modelo real de tu placa y controlador. Los pines usados por el proyecto de PictoBlox proporcionado son:

Elemento	Conexión
Motor 1	Dirección: 7 y 8 PWM: 5
Motor 2	Dirección: 9 y 10 PWM: 6
Servomotor	Señal: pin 3
HC-SR04	TRIG: 12 ECHO: 11
Alimentación	Fuente para motores según el controlador; GND común con Arduino

Seguridad eléctrica

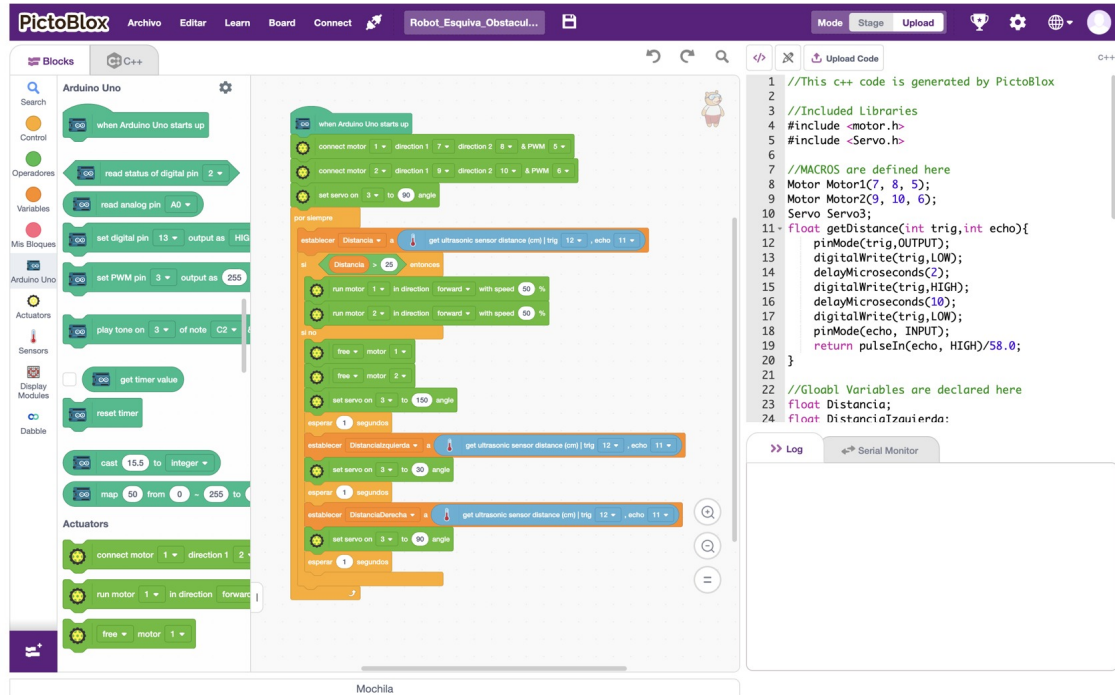
No alimentes los motores desde el pin de 5 V de Arduino. Usa la entrada de potencia del controlador y une las tierras (GND) cuando el módulo lo requiera.

Pruebas unitarias antes de integrar

- Mide cinco distancias conocidas con el ultrasónico.
- Prueba 30°, 90° y 150° del servo sin forzarlo.
- Prueba cada motor por separado y corrige su sentido si es necesario.
- Verifica que el robot se detenga al liberar ambos motores.

Opción 1: programación con bloques

PictoBlox permite construir la lógica visualmente y después consultar el C++ generado. Selecciona Arduino Uno y el modo Upload. La imagen siguiente muestra el proyecto entregado.



Lectura de los bloques

- Configura los dos motores y centra el servo en 90°.
- Mide la distancia frontal con TRIG 12 y ECHO 11.
- Si la distancia es mayor que 25 cm, avanza al 50%.
- Si hay un obstáculo, detén los motores y mide a 150° y 30°.
- Usa las mediciones laterales para ejecutar el giro programado.

Verificación

Durante las pruebas, comprueba que el sentido de giro corresponda con el lado que tiene mayor espacio y ajusta la polaridad o dirección de los motores si el movimiento resulta invertido.

Opción 2: código C++ generado

Versión base proporcionada, normalizada únicamente en formato para que pueda copiarse. Conserva la lógica de PictoBlox y sus pines.

```
// This C++ code is generated by PictoBlox
#include <motor.h>
#include <Servo.h>

Motor Motor1(7, 8, 5);
Motor Motor2(9, 10, 6);
Servo Servo3;

float getDistance(int trig, int echo) {
    pinMode(trig, OUTPUT);
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);
    pinMode(echo, INPUT);
    return pulseIn(echo, HIGH) / 58.0;
}

float Distancia;
float DistanciaIzquierda;
float DistanciaDerecha;

void setup() {
    Servo3.attach(3);
    Servo3.write(90);
}
```

Biblioteca

El encabezado `motor.h` pertenece al entorno/extensión utilizada por PictoBlox. Si se compila en otro entorno, confirma que esa biblioteca esté instalada y sea compatible.

Código C++ generado (continuación)

```
void loop() {
  Distancia = getDistance(12, 11);
  if (Distancia > 25) {
    Motor1.moveMotor(2.55 * 50);
    Motor2.moveMotor(2.55 * 50);
  } else {
    Motor1.freeMotor();
    Motor2.freeMotor();

    Servo3.write(150);
    delay(1000);
    DistanciaIzquierda = getDistance(12, 11);

    Servo3.write(30);
    delay(1000);
    DistanciaDerecha = getDistance(12, 11);

    Servo3.write(90);
    delay(1000);
  }
}
```

Comportamiento del giro

El proyecto incluye la maniobra de giro. Verifica físicamente que ambos motores respondan en el sentido previsto y que el robot vuelva a medir antes de continuar.

Puntos de calibración

- Distancia mínima para detenerse sin tocar el obstáculo.
- Tiempo necesario para completar el giro sobre la superficie elegida.
- Velocidad que evita deslizamientos o lecturas inestables.
- Tiempo de espera del servo antes de cada medición lateral.

Algoritmo completo esperado

INICIO

1. Centrar el sensor.
2. Medir la distancia frontal.
3. Si es mayor que la distancia segura, avanzar.
4. Si no, detenerse y medir izquierda y derecha.
5. Centrar el sensor.
6. Girar hacia el lado con mayor espacio.
7. Si ambos lados están bloqueados, retroceder y girar.
8. Repetir.

Plan obligatorio de pruebas

#	Escenario	Resultado esperado
1	Camino libre	Avanza sin oscilaciones
2	Obstáculo frontal	Se detiene antes del contacto
3	Salida por derecha	Explora y gira a la derecha
4	Salida por izquierda	Explora y gira a la izquierda
5	Callejón sin salida	Retrocede y cambia de dirección
6	Obstáculo angosto	Evita el contacto lateral
7	Lecturas inestables	Filtra o repite la medición
8	Batería reducida	Detecta pérdida de desempeño

En cada prueba registra: distancia, decisión esperada, decisión observada, error, corrección y resultado final.

Evidencias y evaluación

Entregables

- Inventario y diagrama de conexiones adaptado al equipo real.
- Fotografías del proceso de construcción, incluido el material elegido para el chasis.
- Programa en bloques o código C++ como texto copiable.
- Tabla con ocho pruebas, errores y correcciones.
- Video de 60 a 90 segundos con el robot funcionando.
- Página independiente de Google Sites con proceso, evidencias y reflexión.

Criterio	Valor
Diseño y seguridad	15%
Algoritmo y código	20%
Integración de componentes	20%
Funcionamiento autónomo	20%
Pruebas y correcciones	15%
Presentación y documentación	10%

Nombre sugerido

Práctica_22_Robot_esquiva_obstáculos_IACI

Reflexión final

Explica cómo transformó el robot una medición en una decisión, qué calibraciones fueron necesarias, cuál fue el error más importante y qué mejora implementarías en una segunda versión.